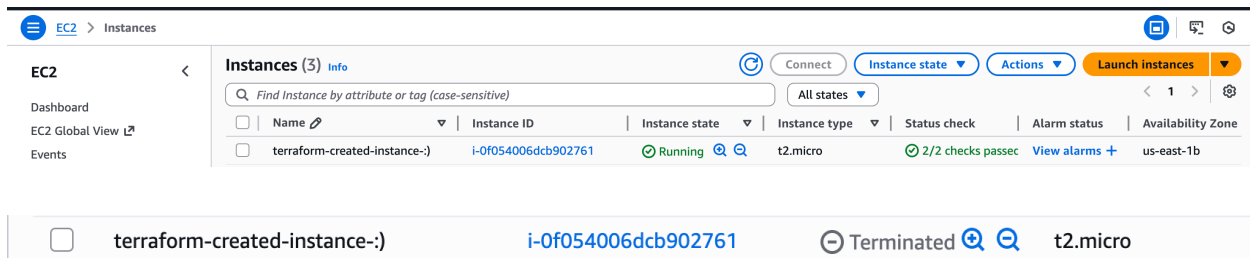


Assignment 2

Part 2:

Successfully deployed the EC2 instance in AWS using terraform. Later the instance was also deleted.

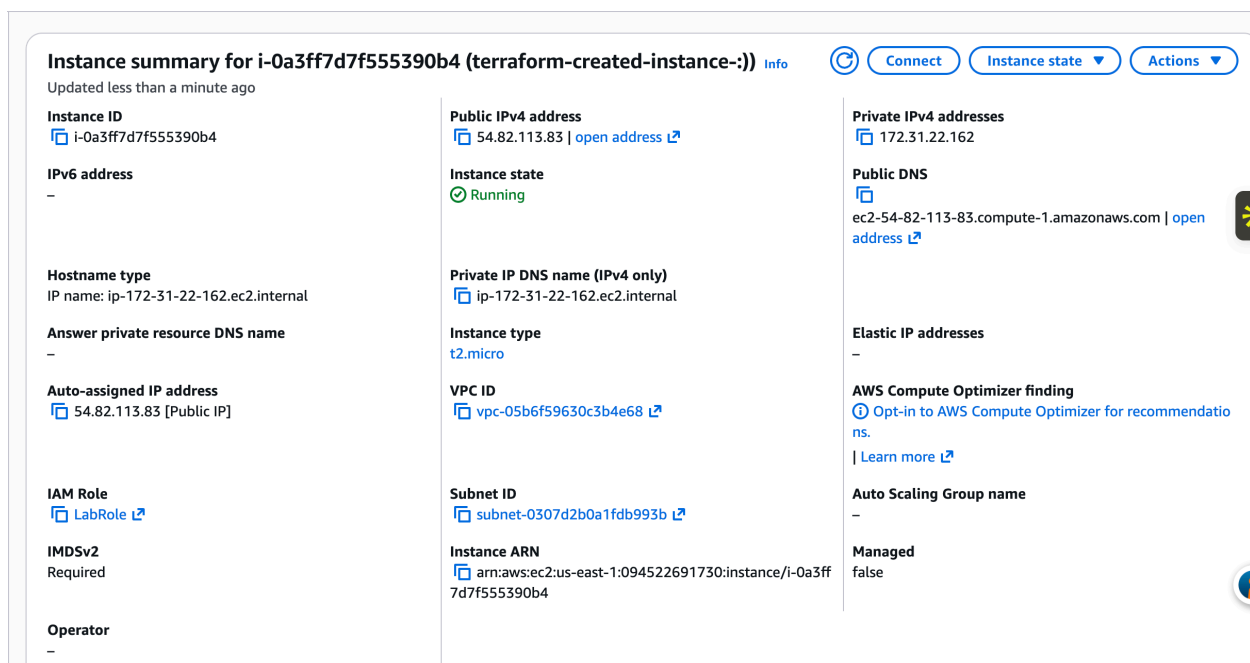


EC2	Instances (3) info	Connect	Instance state	Actions	Launch instances
Dashboard	Find Instance by attribute or tag (case-sensitive)	All states	< 1 >		
EC2 Global View					
Events					
☐	terraform-created-instance-~)	i-0f054006dcb902761	Running	t2.micro	2/2 checks passed View alarms + us-east-1b

Part 3:

Created the docker image for the go application. Installed docker and git in the instance. Loaded and ran the docker image on the instance and had the server up and running. Tested the setup using curl commands from local.

EC2 instance :



Instance summary for i-0a3ff7d7f555390b4 (terraform-created-instance-~) info			Connect	Instance state	Actions
Updated less than a minute ago					
Instance ID	Public IPv4 address		Private IPv4 addresses		
i-0a3ff7d7f555390b4	54.82.113.83 open address		172.31.22.162		
IPv6 address	Instance state		Public DNS		
-	Running		ec2-54-82-113-83.compute-1.amazonaws.com open address		
Hostname type	Private IP DNS name (IPv4 only)		Elastic IP addresses		
IP name: ip-172-31-22-162.ec2.internal	ip-172-31-22-162.ec2.internal		-		
Answer private resource DNS name	Instance type		AWS Compute Optimizer finding		
-	t2.micro		Opt-in to AWS Compute Optimizer for recommendations. Learn more		
Auto-assigned IP address	VPC ID		Auto Scaling Group name		
54.82.113.83 [Public IP]	vpc-05b6f59630c3b4e68		-		
IAM Role	Subnet ID		Managed		
LabRole	subnet-0307d2b0a1fdb993b		false		
IMDSv2	Instance ARN				
Required	arn:aws:ec2:us-east-1:094522691730:instance/i-0a3ff7d7f555390b4				
Operator					
-					

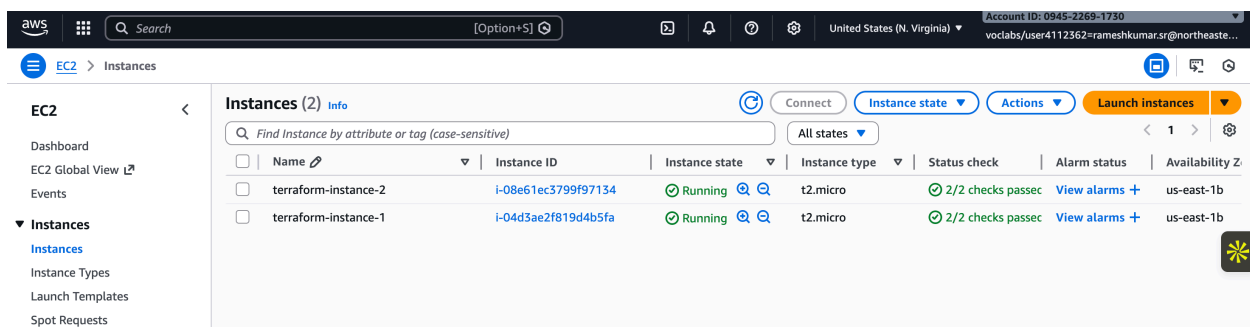
```
(base) srimsansiramesh@Srimansis-Laptop part3 % curl http://54.82.113.83:8080/alb
ums
[
  {
    "id": "1",
    "title": "Blue Train",
    "artist": "John Coltrane",
    "price": 56.99
  },
  {
    "id": "2",
    "title": "Jeru",
    "artist": "Gerry Mulligan",
    "price": 17.99
  },
  {
    "id": "3",
    "title": "Sarah Vaughan and Clifford Brown",
    "artist": "Sarah Vaughan",
    "price": 39.99
  }
]
(base) srimsansiramesh@Srimansis-Laptop part3 %

eb247b45c612: Loading layer 69.94MB/69.94MB
661c822b7477: Loading layer 912B/912B
0ee3e1b12601: Loading layer 58.22MB/58.22MB
Loaded image: go-app:latest
[ec2-user@ip-172-31-22-162 ~]$ docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
go-app               latest             c779b12f5c40       21 hours ago       675MB
amazon/amazon-ecs-agent latest           192298286f1f       13 days ago       120MB
ebs-csi-driver       latest           702835b18362       13 days ago       58.8MB
ecs-service-connect-agent interface-v1       369a1f77771a       4 weeks ago       204MB
amazon/amazon-ecs-pause 0.1.0           9dd4685d3644       11 years ago       702kB
[ec2-user@ip-172-31-22-162 ~]$ docker run -d -p 8080:8080 go-app
b0ba47661d4f01d0a73df529953f7b6a452ff8a84080c749c4741d1bc42b75f5
[ec2-user@ip-172-31-22-162 ~]$ docker ps
CONTAINER ID        IMAGE               PORTS              COMMAND              CREATED            STATUS
b0ba47661d4f    go-app             0.0.0.0:8080->8080/tcp, :::8080->8080/tcp  "/docker-gs-ping"   2 minutes ago     Up 2 minutes
odochoial_ardinghelli
1726a0df0e862    amazon/amazon-ecs-agent:latest  "/agent"           13 minutes ago     Up 13 minutes (healthy)
-agent
[ec2-user@ip-172-31-22-162 ~]$
```

Part 4:

Deployed 2 EC2 instances using terraform. Ran the script from the assignment and attached are the results.

EC2 instances on AWS:



Output of the script:

```

● (base) srinansiramesh@Srimansis-Laptop Assignment2 % python Part4script.py
Starting data test...
Instance 1 response: [
  {
    "id": "1",
    "title": "Blue Train",
    "artist": "John Coltrane",
    "price": 56.99
  },
  {
    "id": "2",
    "title": "Jeru",
    "artist": "Gerry Mulligan",
    "price": 17.99
  },
  {
    "id": "3",
    "title": "Sarah Vaughan and Clifford Brown",
    "artist": "Sarah Vaughan",
    "price": 39.99
  }
]

and...
Instance 2 response: [
  {
    "id": "1",
    "title": "Blue Train",
    "artist": "John Coltrane",
    "price": 56.99
  },
  {
    "id": "2",
    "title": "Jeru",
    "artist": "Gerry Mulligan",
    "price": 17.99
  },
  {
    "id": "3",
    "title": "Sarah Vaughan and Clifford Brown",
    "artist": "Sarah Vaughan",
    "price": 39.99
  }
]

```

```

and adding...
Instance 2 response: {
  "id": "4",
  "title": "The Modern Sound of Betty Carter",
  "artist": "Betty Carter",
  "price": 49.99
}
Instance 1 response: [
  {
    "id": "1",
    "title": "Blue Train",
    "artist": "John Coltrane",
    "price": 56.99
  },
  {
    "id": "2",
    "title": "Jeru",
    "artist": "Gerry Mulligan",
    "price": 17.99
  },
  {
    "id": "3",
    "title": "Sarah Vaughan and Clifford Brown",
    "artist": "Sarah Vaughan",
    "price": 39.99
  }
]

```

```

and...
Instance 2 response: [
  {
    "id": "1",
    "title": "Blue Train",
    "artist": "John Coltrane",
    "price": 56.99
  },
  {
    "id": "2",
    "title": "Jeru",
    "artist": "Gerry Mulligan",
    "price": 17.99
  },
  {
    "id": "3",
    "title": "Sarah Vaughan and Clifford Brown",
    "artist": "Sarah Vaughan",
    "price": 39.99
  },
  {
    "id": "4",
    "title": "The Modern Sound of Betty Carter",
    "artist": "Betty Carter",
    "price": 49.99
  }
]
uhoh... what happened?

```

Observation:

The output demonstrates a data inconsistency problem in distributed systems. Initially, both EC2 instances have the same 3 albums. When a POST request adds "Betty Carter" (id: 4) to Instance 2, only that instance's in-memory data gets updated. Instance 1 remains unaware of the change and still shows only 3 albums.

The "uhoh" moment is that the two instances are now out of sync. In a real-world scenario with load balancing, users could see different data depending on which instance handles their request. The solution is to use a shared database (like RDS or DynamoDB) that both instances connect to, ensuring consistent data across all instances.

Part 5:

Interacted with Claude code to find and fix the bug. This part roughly took me 40 mins.

```
[[ec2-user@ip-172-31-16-153 mystery]$ curl https://qj3wkt6pwd23e67fv5zgvi762u0ijhmk.lambda-url.us-west-2.on.aws/albums
[
  {
    "id": "1",
    "title": "Blue Train",
    "artist": "John Coltrane",
    "price": 56.99
  },
  {
    "id": "2",
    "title": "Jeru",
    "artist": "Gerry Mulligan",
    "price": 17.99
  },
  {
    "id": "3",
    "title": "Sarah Vaughan and Clifford Brown",
    "artist": "Sarah Vaughan",
    "price": 39.99
  }
]
```

There is an endpoint called `/album-count/id` in the `main.go` file mostly. It has an erroneous behaviour -which we need to debug

- Let me read the `main.go` file to understand the endpoint and identify the issue.

- **Read(main.go)**
└ Read 146 lines

- I found the issue! The `postAlbumCount` endpoint at `main.go:105` has a **race condition bug**.

The Problem:

Lines 129-130 show the issue:
`current := albumCounts[index].Count`
`albumCounts[index].Count = current + 1`

The function spawns 10,000 goroutines that all try to increment the counter concurrently, but there's **no synchronization** protecting the shared `albumCounts[index].Count` variable.

What happens:

Multiple goroutines simultaneously:

1. Read the same `Count` value
2. Add 1 to it
3. Write back the result

This causes **lost updates**. For example:

- Goroutine A reads `Count = 5`
- Goroutine B reads `Count = 5`
- Goroutine A writes `Count = 6`
- Goroutine B writes `Count = 6` (overwrites A's update)

Result: Only 1 increment instead of 2.

```
? for shortcuts
[ec2-user@ip-172-31-16-153 mystery]$ curl -X POST https://qj3wkt6pwd23e67fv5zgvi762u0ijhmk.lambda-url.us-west-2.on.aws/album-count/1
{
  "id": "1",
  "count": 10000
}
[ec2-user@ip-172-31-16-153 mystery]$ curl -X POST https://qj3wkt6pwd23e67fv5zgvi762u0ijhmk.lambda-url.us-west-2.on.aws/album-count/1
{
  "id": "1",
  "count": 20000
}
[ec2-user@ip-172-31-16-153 mystery]$ curl -X POST https://qj3wkt6pwd23e67fv5zgvi762u0ijhmk.lambda-url.us-west-2.on.aws/album-count/1
{
  "id": "1",
  "count": 30000
}
```

Mystery Bug: Race Condition in `/album-count/id` Endpoint

The Problem

Claude Code identified a race condition bug in the `postAlbumCount` endpoint at `main.go:105`. The code spawns 10,000 goroutines that all try to increment a counter concurrently, but there's no synchronization (like a mutex) protecting the shared `albumCounts[index].Count` variable.

The Buggy Code (Lines 129-130)

```
current := albumCounts[index].Count
albumCounts[index].Count = current + 1
```

What Happens

Multiple goroutines simultaneously:

1. Read the same `Count` value
2. Add 1 to it
3. Write back the result

Example of Lost Updates

- Goroutine A reads `Count` = 5
- Goroutine B reads `Count` = 5
- Goroutine A writes `Count` = 6
- Goroutine B writes `Count` = 6 (overwrites A's update)

Result: Only 1 increment instead of 2.

The `jsonl` files are attached to the zip folder.